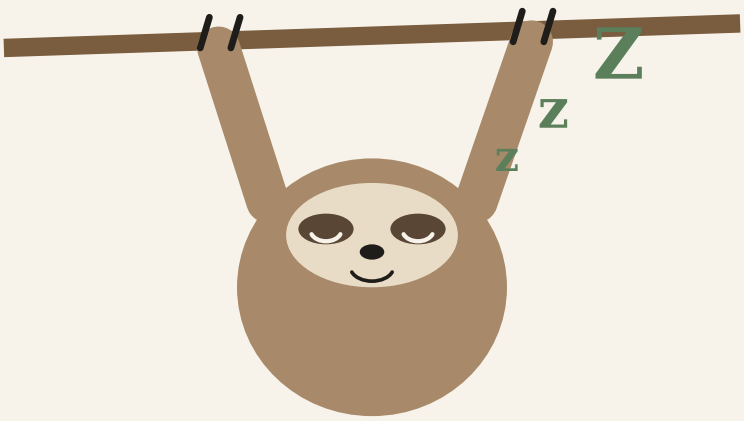


TwoToe

Breach-week triage that runs while you sleep.

FULL BUILD SPEC FOR CLAUDE CODE | WINDOWS EDITION

DEV Hacktoberfest Open-Source AI Challenge, Week 3: "Slow and Steady"

Architecture, contracts, advisory pack, partner tracks, eval, phased prompts, post draft

Deadline: Oct 25, 2026, 11:59 PM PDT (Oct 26, 12:29 PM IST)

Prepared for Soumyadeep Dey (@soumyadeepdey) | Spec version 1.1 (Windows) | Oct 22, 2026

Advisory facts carry VERIFY tags where they may change. Re-check sources before publishing.

Contents

0. Read This First . . . 5

0.1 How to use this PDF with Claude Code . . . 5

0.2 Hard deadline math . . . 5

0.3 Status tags used in this doc . . . 5

1. Mission, Rules, and How Judges Score . . . 7

1.1 The prompt, decoded . . . 7

1.2 Judging criteria and what each means for the build . . . 7

1.3 Rules that can disqualify you . . . 7

1.4 Template sections you must fill . . . 7

2. Product Spec: TwoToe . . . 8

2.1 The story (this is also the post hook) . . . 8

2.2 Who gets to be lazier . . . 8

2.3 User stories (MUST unless marked) . . . 8

2.4 Non-goals (say these out loud in the post) . . . 8

2.5 The brief format (contract) . . . 9

2.6 Confidence tiers (core design idea) . . . 9

3. Ethics and Privacy: Hard Rules . . . 10

4. Architecture . . . 11

4.1 Pipeline overview . . . 11

4.2 Components . . . 11

4.3 Why this architecture wins the judging criteria . . . 12

5. Repo Layout and Tech Stack . . . 13

5.1 Monorepo tree (MUST, create in Phase 0) . . . 13

5.2 Stack . . . 14

5.3 Version pins (VERIFY before Phase 0) . . . 15

5.4 Windows dev environment (do this before Phase 0) . . . 15

6. Data Model (zod contracts) . . . 17

7. The Advisory Pack . . . 19

7.1 What goes in . . . 19

7.2 Pack entries (MUST) . . . 19

7.3 Fixed-version ranges (for the matcher) . . . 20

7.4 Priority function . . . 20

7.5 KEV refresh (SHOULD) . . . 21

8. Scanners and the SystemProbe . . . 22

8.1 The SystemProbe interface (MUST) . . . 22

8.2 Detector specs . . . 22

8.3 Scan scope and performance budget . . . 24
9. Matcher . . . 25
9.1 Rules . . . 25
9.2 Semver helpers (exact behavior) . . . 25
9.3 Matcher truth table (tests MUST cover every row) . . . 25
10. Writer, Prompts, Validator, Template . . . 27
10.1 Ollama client (MUST) . . . 27
10.2 System prompt (MUST, store in prompts.ts, version it) . . . 27
10.3 User prompt template . . . 27
10.4 Few-shot example (SHOULD, include one in the system prompt) . . . 27
10.5 Grounding validator (MUST, the star of the post) . . . 28
10.6 Template writer (MUST) . . . 28
10.7 Line assembly (code, not model) . . . 29
11. Partner Track Integrations . . . 30
11.1 Strategy . . . 30
11.2 Gemma (MUST) . . . 30
11.3 Temporal (MUST) . . . 31
11.4 Sentry Agent Tracing (SHOULD) . . . 32
11.5 .tech domain (SHOULD) . . . 32
11.6 Render or DigitalOcean (SHOULD, pick one) . . . 33
11.7 Mastra "Ask TwoToe" (STRETCH) . . . 33
11.8 ElevenLabs voice brief (STRETCH, opt-in) . . . 33
11.9 Arduino desk sloth (STRETCH, hardware only) . . . 33
12. CLI Spec . . . 34
13. The Morning Card (apps/card) . . . 35
13.1 Design direction . . . 35
13.2 Components . . . 35
13.3 Data loading . . . 35
13.4 Acceptance . . . 35
14. Eval Harness (the numbers that win Writing Quality) . . . 36
14.1 What you will claim in the post . . . 36
14.2 Fixtures (MUST, generated by eval/gen-fixtures.ts) . . . 36
14.3 Metrics and outputs . . . 37
14.4 Human readability check (SHOULD) . . . 37
15. Testing Strategy . . . 38
15.1 Egress guard implementation note . . . 38
16. Phased Build Plan (with Claude Code prompts and gates) . . . 39
Phase 0: Scaffold (MUST) . . . 39

Phase 1: Probe and scanners (MUST) . . . 39
Phase 2: Matcher (MUST) . . . 40
Phase 3: Writer, validator, egress (MUST) . . . 40
Phase 4: Temporal (MUST) . . . 40
Phase 5: Card + deploy (SHOULD) . . . 40
Phase 6: Eval (MUST) . . . 40
Phase 7: Sentry (SHOULD) . . . 40
Phase 8: Ship the story (MUST) . . . 41
Phase 9: Stretch (only if everything above is green) . . . 41

17. The DEV Post (draft skeleton) . . . 42

17.1 Title options (pick after the eval) . . . 42
17.2 Body skeleton . . . 42
17.3 Writing rules for the post . . . 43

18. Demo Video Script (60 to 90 seconds) . . . 44

19. Risk Register . . . 45

20. Final Submission Checklist . . . 46

Appendix A: CLAUDE.md (copy into repo root) . . . 47

Appendix B: Kickoff Prompt (paste into Claude Code) . . . 48

Appendix C: Glossary (for the post, and for future you) . . . 49

Appendix D: Sources (verify before publishing) . . . 50

0. Read This First

This document is the full build spec for **TwoToe**, an entry for the DEV Hacktoberfest Open-Source AI Challenge, Week 3: "Slow and Steady". It is written for two readers at once: you (Soumyadeep) and Claude Code running natively on your Windows machine. Every section is either a decision that is already made, a contract Claude Code must implement exactly, or a checklist you tick before moving on.

> **The one-sentence product:** a local, open-weight agent that does breach-week triage overnight while you sleep, and at 7 a.m. hands you two lines and a short rotate/patch checklist. Code owns the facts. The model only phrases them. No secret value ever leaves your machine.

0.1 How to use this PDF with Claude Code

1. Create a fresh repo (the challenge requires a project started inside the Oct 19 to Oct 25 window). Name it `twotoe`.
2. Copy this PDF to `docs/SPEC.pdf` and the companion markdown to `docs/SPEC.md`. Claude Code can read both, but the markdown is easier for it to grep.
3. Copy the `CLAUDE.md` from Appendix A into the repo root. Claude Code loads it automatically every session.

> **Windows note:** run Claude Code natively on Windows (PowerShell or Windows Terminal), not inside WSL2. TwoToe has to see your real Windows machine: the Node, Python, PHP, and Plex that WSL cannot see. Section 5.4 has the full Windows setup.

4. Start Claude Code in the repo and paste the **Kickoff Prompt** from Appendix B.
5. Work phase by phase (Section 16). Each phase ends with an **acceptance gate**. Do not let Claude Code start the next phase until the gate passes. Gates are where you catch drift, not at 11 p.m. on Oct 25.
6. Keep the Claude Code session transcript. The DEV template has an optional "My Agent Session" section, and judges like it.

0.2 Hard deadline math

Event	Pacific (PDT)	Kolkata (IST)
Challenge opened	Oct 19	Oct 19
You start building (this doc)	Oct 22	Oct 22 evening
Submissions close	**Oct 25, 11:59 PM**	**Oct 26, 12:29 PM**
Your personal freeze target	Oct 25, 6:00 AM	Oct 25, 6:30 PM
Winners announced	week of Oct 26	week of Oct 26

Plan to freeze code a full day early in your timezone. The post is weighted more heavily than the code, and the post needs real eval numbers, a GIF, and a video, all of which depend on frozen code.

0.3 Status tags used in this doc

Tag	Meaning for Claude Code
MUST	Required for the MVP. Build it, test it, gate on it.
SHOULD	Build it if the phase is on time.
STRETCH	Only after every MUST and SHOULD is green and the post draft exists.
VERIFY	A fact that may have changed since Oct 22, 2026. Claude Code must check the linked primary source before hardcoding it, and record the check date in the advisory pack.

Path convention: `~` in this doc means `os.homedir()`. On your machine that is `C:\Users\<you>`, so `~/.twotoe` is `C:\Users\<you>\.twotoe`. Code must always build paths with `path.join` and never hardcode `/` or `\`.

1. Mission, Rules, and How Judges Score

1.1 The prompt, decoded

The base prompt for all five weeks: build something with open-source AI at its core (open-weight model, open-source agent harness, local inference, or a mix), where the open pieces are what make the project work. Week 3 adds: build something "that rewards laziness". Help someone relax, automate a task they dread, or make life easier. DEV's own example list includes a local agent that reads things overnight and hands you a two-line morning summary. TwoToe is that example pointed at the most dreaded task of the week: figuring out whether ten scary security headlines actually affect *you*.

1.2 Judging criteria and what each means for the build

Criterion	Weight	What wins it	Where in this spec
Writing Quality	Heaviest	Specific hook, real numbers, honest failure section, clean structure, personality	Section 15
Relevance to Prompt and Theme	High	Literal laziness: two lines, one command, runs while you sleep	Sections 2, 9
Creativity	Medium	Not another group-chat summarizer. Breach-week triage with a privacy argument	Section 2
Technical Execution	Medium	Grounding validator, deterministic matcher, Temporal durability, eval harness	Sections 5 to 12
Use of Partner Technology	Optional	Genuine, load-bearing use only. Tick-box integrations read as tick-box	Section 11

The FAQ line to tattoo on the monitor: a great project with a thin write-up won't win.

1.3 Rules that can disqualify you

- The project and its repo must be started and finished inside the challenge window. Start a new repo. Do not copy TrailTruth code without crediting it in the README.
- Commits after the deadline must be noted in the README, or you risk disqualification. Simplest rule: tag `v1.0-submission` before the deadline and do not push after it until winners are out.
- Significant reuse of other open-source code must be credited.
- One submission per challenge. English only for prize eligibility. You must be 18+.
- Tags: `#devchallenge` and `#hf26challenge` (the template adds them; do not delete them).
- DEV says a submission can win once per challenge and it aims to celebrate as many different builders as possible across the month. You won Week 1. Assume a headwind on crowded partner categories and aim for **Overall** on writing quality, with Gemma, Temporal, and Sentry as genuine secondary shots.

1.4 Template sections you must fill

What I Built, Demo, Code, How I Built It, Why Does Open Innovation Matter?, My Agent Session (optional), The Lazy Payoff (optional), Prize Categories. The two optional sections are where you beat people who skip them. You will not skip them.

2. Product Spec: TwoToe

2.1 The story (this is also the post hook)

On Oct 21, 2026 a viral post claimed "the internet broke today" and listed ten incidents. Eight are real, some numbers are inflated (eMudhra's "60M certs at risk" is a lifetime issuance count; eMudhra's own Bugzilla estimate is about 30,000 certs needing migration), and several did not happen "today". The honest question every developer had was not "is the internet broken" but "**which of these hit my machine, and what do I do before breakfast?**" Answering that by pasting lockfiles and `.env` names into a cloud chatbot is exactly backwards, because half the week's incidents were credential leaks. TwoToe answers it locally, overnight, in two lines.

2.2 Who gets to be lazier

- Solo devs and students with five side projects, two Python venvs, a Railway hobby app, and Plex on the home server.
- Freelancers and small agencies who maintain client sites on PHP and Node and cannot read every advisory.
- Anyone who saw the viral post and felt the "I should check... later" dread.

2.3 User stories (MUST unless marked)

1. As a dev, I run `twotoe sleep` once and every night at 3 a.m. it scans my configured project roots and machine, so I never have to remember.
2. As a dev, when I wake up I read **exactly two lines**: what needs me today and confirmation that everything else is fine.
3. As a dev, I get an ordered checklist (max 7 items) where each item says what to do, why, where it was found, and links to the vendor advisory.
4. As a dev, I can trust that no secret value was read, logged, or sent anywhere, and the card shows a network egress ledger proving it.
5. As a dev, when the model says something, every version number, CVE ID, and vendor name in it was matched by code first. If the model invents one, it gets thrown out.
6. As a dev, I can run `twotoe brief --no-llm` and get a fully templated brief with zero model calls (fallback and baseline for the eval).
7. As a judge, I can open a live demo link with an example brief and click "See an Example" without installing anything.
8. (SHOULD) As a dev, I get a Windows toast notification at 7 a.m. with the two lines.
9. (STRETCH) As a dev, I can hear the brief read aloud while brushing my teeth.
10. (STRETCH) As a dev, I can ask a follow-up ("why does the Node one matter?") and get an answer grounded only in matched findings.

2.4 Non-goals (say these out loud in the post)

- Not a vulnerability scanner replacement (no full SCA, no SBOM ingestion, no CVE database mirror). It checks a curated, dated advisory pack of the incidents that matter this week, plus CISA KEV.
- Not an exploit tool. No PoCs, no payload reproduction, no active probing of remote hosts.
- Not a secrets manager. It reads key **names**, never values, and it never rotates anything for you. It hands you links.
- Not a dashboard you are supposed to stare at. The product is two lines.

2.5 The brief format (contract)

```
Line 1:  "{N} thing(s) need you today: {action 1}; {action 2}."
         or "Nothing needs you today."
Line 2:  "Everything else: you're fine. Checked {X} things in {T}s, 0 secrets left this machine."
Checklist: up to 7 items, ordered by priority, each with:
         action (imperative, <= 90 chars), why (<= 140 chars),
         evidence (detector + location), confidence, source URL.
FYI:     up to 3 awareness lines that never count as actions
         (for example the SBI data exposure for users with region=IN).
```

Line 1 lists at most 2 actions inline. If there are more, it ends with "+{k} more in the checklist." Each line is capped at 160 characters. Counts and timings in Line 2 are filled by code, never by the model.

2.6 Confidence tiers (core design idea)

Tier	Meaning	Example
`definite`	A version or file was matched exactly	`litellm==1.107.0` installed in `~\code\x\.venv`
`likely`	Strong signal, but the vulnerable condition cannot be proven locally	`RAILWAY_TOKEN` key name present plus `railway.toml` in repo
`possible`	Weak signal, user must confirm	`opencode` CLI installed (breach was Console users with linked providers)
`fyi`	Not machine-checkable	SBI customer data exposure notice

Only `definite` and `likely` count toward Line 1. `possible` items appear in the checklist with a "confirm" verb ("Confirm whether you linked API keys in OpenCode Console; if yes, revoke them."). `fyi` items go to the FYI block.

3. Ethics and Privacy: Hard Rules

These rules are non-negotiable. Put them in `CLAUDE.md`. Every PR Claude Code makes must respect them, and there are tests for each.

1. **Never read secret values.** The env scanner reads a file line by line, splits on the first `=`, keeps the left side, and discards the right side immediately (it is never assigned to a variable that outlives the loop iteration, never logged, never put in an error message). Test: a fixture `.env` with a canary value `TWOTOE_CANARY_9f3a` must never appear in any output, log, trace, or brief.
2. **Never execute or open IOC payloads.** For LiteLLM 1.107.0 indicators (`cook.pth`, `cook.py`, suspicious `sitecustomize.py`), check existence, size, and SHA-256 only. Never `import`, never `cat`, never run Python with that site-packages on the path. Use `fs.stat` and a streamed hash.
3. **Never run package managers that execute code.** No `pip install`, no `npm install`, no `python -c` that imports user packages. Python version discovery uses dist-info folder names on disk, not `pip list` (pip itself is fine to *not* touch). Exception: `python3 -c "import certifi, sys; print(certifi.where())"` is NOT allowed either, because importing triggers `.pth` files. Find certifi bundles by globbing `site-packages/certifi/cacert.pem`.
4. **Network allowlist.** All outbound traffic goes through a guard that only allows: `127.0.0.1` / `localhost` (Ollama on 11434, Temporal on 7233/8233, the card server), `www.cisa.gov` (KEV JSON feed), and, only if the user opted in, the Sentry ingest host. Everything else throws `EgressBlockedError`. The guard records every request (host, method, bytes out) into the egress ledger shown on the card.
5. **Offline must work.** `--offline` skips the KEV refresh and uses the bundled pack. The brief must still be produced.
6. **Defensive only.** No exploit code, no PoC, no scanning of hosts you do not own. The checklist links to vendor advisories as the source of truth.
7. **Synthetic data for demos.** The public demo, screenshots, GIF, and video use fixtures, never your real machine paths. Real-machine results appear in the post only as aggregate counts.
8. **Telemetry is opt-in and content-free by default.** Sentry spans carry counts, durations, model name, and token usage. Prompt and completion text are only attached in `demo` mode, which runs on fixtures.
9. **Honest output.** TwoToe can be wrong. Every checklist item links to the vendor. The card footer says so.

4. Architecture

4.1 Pipeline overview



The load-bearing idea: the model sits between two deterministic walls. The left wall (matcher) decides *what is true*. The right wall (validator) rejects anything the model said that the left wall did not establish. The model's only job is to turn structured findings into calm, human sentences.

4.2 Components

Component	Package	Responsibility	AI?
SystemProbe	`@twotoe/core`	Reads machine facts: Node, PHP, Plex, GitLab versions, trust stores, docker images, CLI presence	No
Project scanners	`@twotoe/core`	Walk project roots: lockfiles, Python venvs, Dockerfiles, `.env` key names, IOC files	No
Advisory pack	`@twotoe/core`	Versioned JSON of verified incidents + KEV snapshot	No
Matcher	`@twotoe/core`	Inventory x Advisories -> Findings with confidence and priority	No
Writer	`@twotoe/writer`	Ollama client, prompts, structured JSON output	**Yes**
Validator	`@twotoe/writer`	Grounding check, length check, banned-claim check	No
Templater	`@twotoe/writer`	Fully deterministic brief (fallback and eval baseline)	No
Egress guard	`@twotoe/core`	Allowlisted fetch + ledger	No
CLI	`@twotoe/cli`	`scan`, `brief`, `sleep`, `card`, `eval`, `doctor`, `pack`, `explain`	No
Worker	`@twotoe/worker`	Temporal workflow + activities + schedule	No
Card	`apps/card`	React/Tailwind morning card, reads brief JSON	No

Component	Package	Responsibility	AI?
Eval	`eval/`	Fixtures, ground truth, metrics, results tables	Runs the writer

4.3 Why this architecture wins the judging criteria

- **Technical Execution:** two deterministic walls around a small model is a pattern judges recognize as mature. The Week 1 overall winner (Paper Trail) won on exactly this kind of honesty ("every line on a card is either checked or quoted").
- **Open innovation:** the model is local, the facts are local, the secrets are never read. You can explain why a cloud model is the wrong tool in one sentence.
- **Laziness:** one command to install the schedule, two lines to read.

5. Repo Layout and Tech Stack

5.1 Monorepo tree (MUST, create in Phase 0)

```
twotoe/
  CLAUDE.md
  README.md
  LICENSE                       (MIT)
  package.json                  (pnpm workspaces root, scripts)
  pnpm-workspace.yaml
  tsconfig.base.json
  .nvmrc                        (24.22.1 or newer patched LTS, see 5.3)
  .env.example                  (only TWOTOE_* config keys, no secrets)
  docs/
    SPEC.pdf
    SPEC.md
    post/                       (DEV post drafts, images, GIF)
  packages/
    core/
      src/
        schemas.ts              (zod: Inventory, Advisory, Finding, Brief...)
        pack/
          advisories.json       (bundled advisory pack, dated)
          kev-snapshot.json     (CISA KEV snapshot at build time)
          loadPack.ts
          refreshKev.ts
        probe/
          SystemProbe.ts        (interface)
          RealProbe.ts          (Windows first, plus macOS + Linux)
          FixtureProbe.ts       (reads probe.json for eval/demo)
        scanners/
          envKeys.ts
          nodeLockfiles.ts
          pythonEnvs.ts
          dockerfiles.ts
          iocFiles.ts
          trustStores.ts
          projectMarkers.ts     (railway.toml, opencode presence, etc.)
        match/
          semverUtil.ts
          matchers/             (one file per advisory id)
          matchAll.ts
          priority.ts
        egress/
          guard.ts
          ledger.ts
        index.ts
      test/
```

```
    writer/
      src/
        ollama.ts
        prompts.ts
        writeBrief.ts
        validate.ts
        template.ts
        tokens.ts              (allowed-token extraction)
      test/
    cli/
      src/
        index.ts               (commander)
        commands/*.ts
        notify.ts              (Windows toast via node-notifier)
        taskScheduler.ts       (Windows Task Scheduler install/uninstall)
    worker/
      src/
        workflows.ts
        activities.ts
        worker.ts
        schedule.ts
        tracing.ts             (Sentry spans)
      scripts/
        crash-test.ts          (cross-platform; taskkill on Windows)
  apps/
    card/
      index.html
      src/
        main.tsx
        App.tsx
        components/*.tsx
        example/brief.example.json
      tailwind.config.ts
  eval/
    fixtures/
      f01-litellm-bad/ ... f24-clean-tricky/
        probe.json
        expected.json
        project/ (synthetic files)
    gen-fixtures.ts
    run-eval.ts
    RESULTS.md                 (auto-generated tables for the post)
```

5.2 Stack

Layer	Choice	Why
Language	TypeScript (strict) everywhere	Your strongest language; one toolchain
Runtime	Node.js, a **patched** line (see 5.3)	Dogfood: the tool should not run on a vulnerable Node
Package manager	pnpm workspaces	Fast, strict, good monorepo story
Validation	zod	Schemas double as JSON schema for Ollama structured output (via `zod-to-json-schema`)
Versions	`semver`	Range matching
Lockfile parsing	`@yarnpkg/lockfile` for yarn v1, `yaml` for pnpm, native JSON for npm	No network, no install
CLI	`commander`, `picocolors`, `ora`	Familiar, small
Local model	Ollama for Windows + Gemma 4 E4B (default), configurable	Gemma featured category; runs on a normal Windows laptop, faster with an NVIDIA/AMD GPU
Orchestration	Temporal TypeScript SDK, local dev server	Durable nightly runs; Temporal partner category; you already know it from Week 1
Tracing	`@sentry/node` with AI agent span conventions	Sentry Agent Tracing partner category

Layer	Choice	Why
UI	Vite + React + TypeScript + Tailwind	Your home turf
Tests	Vitest	Fast, TS-native
Hosting (demo)	Render static site **or** DigitalOcean App Platform (pick one, Section 11)	Credits available via Hacktoberfest promos
Domain	Free `.tech` domain from promos, e.g. `twotoe.tech`	Partner category + nicer demo link

5.3 Version pins (VERIFY before Phase 0)

Thing	Pin	Note
Node	`>= 24.22.1` (or `>= 22.23.4` / `>= 26.12.1`)	These are the Oct 21, 2026 security releases. VERIFY at nodejs.org/en/blog/vulnerability/october-2026-security-releases
Ollama	latest stable	Needs structured output support (`format` accepts a JSON schema)
Model tag	Gemma 4 E4B, exact tag from ollama.com/library	VERIFY the tag. Fallback env `TWOTOE_MODEL=gemma3:4b` if Gemma 4 is unavailable
Temporal CLI	latest	Windows: download `temporal.exe` from the official Temporal CLI install page (VERIFY whether winget/scoop packages exist), put it on PATH, then `temporal server start-dev`
Temporal TS SDK	latest `@temporalio/*` (all four packages same version)	Mixed versions break the worker
Sentry	latest `@sentry/node` v9+ (or current major)	VERIFY current AI agent monitoring docs before writing spans

Rule for Claude Code: run `pnpm view <pkg> version` for each dependency and pin exact versions in `package.json`. Do not use `latest` or `^` in the final submission.

5.4 Windows dev environment (do this before Phase 0)

You are building and demoing on Windows. That is a strength, not a problem: most breach-triage tools assume macOS or Linux, so "it works on the laptop most students actually own" is a line for the post. But Windows has sharp edges, so set it up deliberately.

Install list

Tool	How	Check
Node (patched LTS)	Official installer from nodejs.org, or `fnm` / `nvm-windows`. Must be >= 24.22.1 or >= 22.23.4	`node -v`
pnpm	`corepack enable` then `corepack prepare pnpm@latest --activate` (run terminal as admin once if corepack complains)	`pnpm -v`
Git for Windows	git-scm.com. Claude Code on Windows uses Git Bash under the hood (VERIFY current Claude Code Windows requirements)	`git --version`
Ollama for Windows	ollama.com installer. Runs as a tray app on `127.0.0.1:11434`	`ollama list`, then `ollama ps` while a model runs to see GPU vs CPU
Temporal CLI	Download `temporal.exe` (see 5.3), add its folder to PATH	`temporal --version`

Tool	How	Check
Docker Desktop (optional)	Only if you already use it. TwoToe reads image tags if the daemon answers	`docker images`
PHP (optional)	Only if you already have it. Do not install PHP just for the demo; fixtures cover it	`php -v`

One-time repo settings

```
git config --global core.longpaths true      # deep node_modules + fixture paths
git config --global core.autocrlf false
```

Add a `.gitattributes` at the repo root:

```
* text=auto eol=lf
*.ps1 text eol=crlf
*.png binary
*.gif binary
```

If PowerShell blocks scripts (`pnpm.ps1 cannot be loaded`), run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

Windows rules for Claude Code (also in CLAUDE.md)

1. **No bash-only scripts.** Every script is TypeScript run with `tsx` (works everywhere). No `.sh` files, no `rm -rf`, no `&&` chains that rely on Unix tools. Use `rimraf`, `cross-env`, `shx` only if truly needed.
2. **Paths:** `path.join` / `path.resolve` everywhere. For globbing use `fast-glob` with forward slashes (convert with `fg.convertPathToPattern`). Compare paths case-insensitively on Windows.
3. **Spawning:** native `.exe` programs (`node`, `php`, `docker`, `git`) work with `execFile`. Tools installed as `.cmd` shims (`pnpm`, `npm`, `railway`, `opencode` when installed via npm) do **not**. Use `cross-spawn` (or `execa`) for those, still with an argument array, never a shell string.
4. **PowerShell calls** (trust store, Plex file version, toast fallback, voice): always `powershell.exe -NoProfile -NonInteractive -Command <script>` with any user-derived value passed through an environment variable, never interpolated into the script text.
5. **Redaction:** redact `C:\Users\<name>` (and `/Users/<name>`, `/home/<name>`) to `~` in every output, log, trace, and brief.
6. **File locks:** Windows Defender and editors can hold files open. Every `fs` read in scanners is wrapped so `EBUSY`, `EPERM`, and `EACCES` skip the file and count it in `brief.meta` instead of crashing the scan.
7. **Line endings:** every text parser accepts CRLF.

Hardware reality check

Run `ollama run <gemma e4b tag> "hi"` and then `ollama ps`. If it shows `100% CPU`, you have no usable GPU for Ollama, and that changes the plan:

- Nightly use is still fine (a minute per brief at 3 a.m. bothers nobody).
- For the eval, use Gemma 4 E2B for bulk runs, drop to `--runs 3`, and run the E4B comparison on a subset of 8 fixtures. Note this honestly in the post. "Runs on a CPU-only student laptop" is a feature.
- Close Chrome and games during the eval. Report your laptop's CPU, RAM, and GPU in the post next to the latency numbers.

6. Data Model (zod contracts)

Claude Code must implement these schemas exactly in `packages/core/src/schemas.ts`. Everything downstream depends on them.

```
import { z } from "zod";

export const Confidence = z.enum(["definite", "likely", "possible", "fyi"]);
export const Severity = z.enum(["critical", "high", "medium", "info"]);

// ---------- Inventory (facts collected, no judgment) ----------
export const InventoryItem = z.discriminatedUnion("kind", [
  z.object({ kind: z.literal("runtime"), name: z.enum(["node", "php", "python"]),
             version: z.string(), source: z.string() }),          // source: "node --version",
                 "Dockerfile:3"
  z.object({ kind: z.literal("package"), ecosystem: z.enum(["npm", "pypi"]),
             name: z.string(), version: z.string(), source: z.string() }),
  z.object({ kind: z.literal("app"), name: z.enum(["plex", "gitlab"]),
             version: z.string(), source: z.string() }),
  z.object({ kind: z.literal("dockerImage"), image: z.string(), tag: z.string(),
             source: z.string() }),
  z.object({ kind: z.literal("envKey"), key: z.string(), file: z.string() }),  // NAME ONLY
  z.object({ kind: z.literal("iocFile"), filename: z.string(), dir: z.string(),
             size: z.number(), sha256: z.string() }),
  z.object({ kind: z.literal("trustAnchor"), store: z.string(), subject: z.string(),
             present: z.boolean() }),
  z.object({ kind: z.literal("marker"), marker: z.string(), path: z.string() }), // railway.toml,
      opencode cli
]);
export type InventoryItem = z.infer<typeof InventoryItem>;

export const Inventory = z.object({
  scannedAt: z.string().datetime(),
  roots: z.array(z.string()),
  durationMs: z.number(),
  items: z.array(InventoryItem),
  checksRun: z.number(),               // used for "Checked X things" in line 2
});

// ---------- Advisory pack ----------
export const Advisory = z.object({
  id: z.string(),                      // "litellm-1.107.0"
  title: z.string(),
  vendor: z.string(),
  cves: z.array(z.string()),
  severity: Severity,
  kev: z.boolean(),
  kevDueDate: z.string().nullable(),
  disclosedAt: z.string(),             // ISO date
```

```
  sourceUrls: z.array(z.string().url()).min(1),
  verifiedAt: z.string(),              // when a human/Claude last checked the source
  summary: z.string(),                 // one factual sentence, used by template + allowed tokens
  action: z.string(),                  // canonical imperative action
  check: z.object({
    detector: z.string(),              // name of matcher
    params: z.record(z.any()),
  }),
  regions: z.array(z.string()).optional(), // e.g. ["IN"] for FYI items
});

// ---------- Findings (matcher output) ----------
export const Finding = z.object({
  id: z.string(),                      // `${advisoryId}:${hash(evidence)}`
  advisoryId: z.string(),
  confidence: Confidence,
  severity: Severity,
  priority: z.number(),                // lower = more urgent, see 7.4
  evidence: z.array(z.object({ detector: z.string(), location: z.string(), detail: z.string() })),
  allowedTokens: z.array(z.string()),  // every number/CVE/vendor/version the writer may mention
});

// ---------- Brief (writer output after validation) ----------
export const ChecklistItem = z.object({
  findingId: z.string(),
  action: z.string().max(90),
  why: z.string().max(140),
});
export const Brief = z.object({
  date: z.string(),
  line1: z.string().max(160),
  line2: z.string().max(160),
  checklist: z.array(ChecklistItem).max(7),
  fyi: z.array(z.string().max(160)).max(3),
  meta: z.object({
    mode: z.enum(["llm", "template", "llm-fallback"]),
    model: z.string().nullable(),
    attempts: z.number(),
    validatorRejections: z.array(z.string()),
    durationMs: z.number(),
    egress: z.array(z.object({ host: z.string(), method: z.string(), bytesOut: z.number() })),
  }),
});
```

Rule: the writer model receives `Finding[]` serialized with only these fields: `id, advisoryId, confidence, severity, title, vendor, action, summary, evidence[].location (redacted to project-relative paths), allowedTokens`. It never receives raw file contents, absolute home paths, or env key lists beyond the matched ones.

7. The Advisory Pack

7.1 What goes in

Only incidents verified against a primary or reputable source, each with a `verifiedAt` date. This is the pack as of Oct 22, 2026, built from the fact-check. Every VERIFY field must be re-checked by Claude Code via the source URL before the `v1.0-submission` tag, and again on the day you publish.

7.2 Pack entries (MUST)

id	What	Severity / KEV	Detector	Action
`litellm-1.107.0`	LiteLLM PyPI 1.107.0 was malicious (credential stealer + hourly backdoor). Quarantined same day. Last safe: 1.106.2	critical / no	`pypiExactVersion` + `iocFiles`	Remove the env, reinstall pinned 1.106.2, rotate every cloud/SSH/CI/AI key that env could read
`railway-2026-10-21`	Railway disclosed unauthorized access to encrypted env vars, some decrypted	critical / no	`railwayMarkers` + `envKeys(RAILWAY_*)`	Roll every env var, API key, registry credential stored in Railway
`opencode-console-2026-10-17`	OpenCode Console legacy system accessed (disclosed Oct 17); provider API keys of affected users exposed	high / no	`opencodePresence` (possible only)	Confirm whether you linked provider keys in OpenCode Console; if yes, revoke them
`php2shell-2026`	CVE-2026-109618, CVE-2026-109619, CVSS v4 9.2 unauth RCE; fixed in 8.5.12, 8.4.27, 8.3.36, 8.2.35; CISA says take 8.1 and older offline	critical / no (VERIFY)	`runtimeVersion(php)` + `dockerTag(php)`	Upgrade PHP to the fixed patch for your line; take 8.1 or older offline
`nodejs-2026-10-21`	10 fixes incl. CVE-2026-111497 Brotli OOB write (High) and a TLS hostname-validation bypass; fixed in 22.23.4, 24.22.1, 26.12.1	high / no	`runtimeVersion(node)` + `dockerTag(node)` + `nvmrc`	Upgrade Node to the fixed release for your line
`gitlab-cve-2026-110264`	CVSS 10.0, regex compiler integer overflow, self-managed CE/EE back to 7.0	critical / no (VERIFY)	`appVersion(gitlab)` + `dockerTag(gitlab/gitlab-ce)` or `dockerTag(gitlab/gitlab-ee)`	Upgrade self-managed GitLab to the patched release
`gitlab-cve-2026-97229`	CVSS 10.0, patched Oct 12, needs ability to commit CI config to a public project	critical / no (VERIFY)	same as above	Same upgrade

id	What	Severity / KEV	Detector	Action
`plex2shell-2026`	CVE-2026-96652, -96654, -96655, -96656 actively exploited (miners, proxy malware); fixed in PMS 1.43.3.10861	critical / **yes**, due Oct 23 and Nov 3	`appVersion(plex)` + `dockerTag(plexinc/pms-docker)`	Update Plex Media Server to 1.43.3.10861 or newer
`emudhra-weak-ca-keys`	8 intermediate CA keys were trivially recoverable; root distrusted by browsers; non-browser clients (curl, Python, Node) may still trust it	high / no	`trustStores(emSign ECC Root CA - G3)`	Upgrade Node (bundled store), update certifi in each venv, update OS
`sbi-2026-10-21`	SBI disclosed 3,897 customers' PII incl. loan/lead metadata found in dark-web data	info / no	`region(IN)` FYI only	Expect targeted phishing; SBI never asks for OTP or PIN
`kev-recent`	Any CISA KEV entry added in the last 7 days whose vendor/product matches an inventory item	varies / yes	`kevProductMatch`	Patch per CISA due date

The Rajasthan Police item is **not** in the pack. It is an unverified claim, not actionable, and including it would undermine the honesty story. Mention that choice in the post.

7.3 Fixed-version ranges (for the matcher)

```
{
  "nodejs-2026-10-21": {
    "fixed": { "22": "22.23.4", "24": "24.22.1", "26": "26.12.1" },
    "unsupportedBelowMajor": 22,
    "note": "VERIFY whether any other line received a release"
  },
  "php2shell-2026": {
    "fixed": { "8.5": "8.5.12", "8.4": "8.4.27", "8.3": "8.3.36", "8.2": "8.2.35" },
    "takeOfflineAtOrBelow": "8.1"
  },
  "plex2shell-2026": { "fixedAtLeast": "1.43.3.10861" },
  "litellm-1.107.0": { "badExact": ["1.107.0"], "lastSafe": "1.106.2",
                       "iocFilenames": ["cook.pth", "cook.py"],
                       "suspiciousIfPresent": ["sitecustomize.py"] },
  "gitlab-cve-2026-110264": { "fixed": "VERIFY from GitLab patch release post", "affectedFromMajor": 7
      },
  "emudhra-weak-ca-keys": { "subjects": ["emSign ECC Root CA - G3"] }
}
```

7.4 Priority function

```
priority = base(severity) - kevBonus - confidenceBonus
base: critical=100, high=200, medium=300, info=900
kevBonus: 50 if kev, plus 25 more if kevDueDate within 3 days
confidenceBonus: definite=30, likely=15, possible=0, fyi=-500 (pushes to the bottom)
Tie-break: credential-exposure advisories (railway, opencode, litellm) before patch advisories,
because a leaked key is being used right now and a patch gap is only being scanned for.
```

7.5 KEV refresh (SHOULD)

`refreshKev()` fetches `https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json` through the egress guard, keeps entries with `dateAdded` in the last 14 days, and stores a snapshot. The `kev-recent` matcher compares `vendorProject` and `product` (case-insensitive, normalized) against inventory names (node, php, plex, gitlab, plus npm/pypi package names). Keep it conservative: only exact normalized product matches produce `likely`; fuzzy matches produce `possible`. VERIFY the feed URL and field names before coding.

8. Scanners and the SystemProbe

8.1 The SystemProbe interface (MUST)

Every machine-level fact goes through one interface, so the eval and the public demo can swap in fixtures without touching scanner code.

```
export interface SystemProbe {
  platform(): Promise<"win32" | "darwin" | "linux" | "other">;
  nodeVersion(): Promise<string | null>;          // process.version of the *user's* default node (see
      8.2)
  phpVersion(): Promise<string | null>;
  plexVersion(): Promise<string | null>;
  gitlabVersion(): Promise<string | null>;
  dockerImages(): Promise<Array<{ repo: string; tag: string }>>;
  trustStoreHas(subject: string): Promise<Array<{ store: string; present: boolean }>>;
  commandExists(cmd: string): Promise<boolean>;
  homeMarkers(): Promise<string[]>;               // e.g. "~\.railway" exists (existence only)
}
```

`RealProbe` uses `execFile` for native executables and `cross-spawn` for `.cmd` shims (see 5.4), always with an argument array, never `exec`, never a shell string, with a 5 second timeout. It branches on `process.platform`, with **Windows implemented and tested first**; macOS and Linux branches are SHOULD (nice for readers of the post, not needed for your demo). `FixtureProbe` reads `probe.json`.

8.2 Detector specs

8.2.1 Node runtime (MUST)

- `execFile("node", ["--version"])` through the user's PATH, plus every `.nvmrc`, `.node-version`, `package.json#engines.node`, and Dockerfile `FROM node:<tag>` found under project roots.
- Normalize: strip leading `v`. For Docker tags, parse leading semver (`node:24.22.0-alpine` -> `24.22.0`; `node:24-alpine` -> major only, confidence `likely` with detail "floating tag, rebuild to pull the patched image").
- Also report `process.version` of TwoToe itself and refuse to run (warn loudly) if it is below the fixed version. Dogfood moment for the post.

8.2.2 PHP runtime (MUST)

- `execFile("php", ["-v"])`, parse `PHP (\d+\.\d+\.\d+)`.
- Dockerfile `FROM php:<tag>` and `docker-compose.yml image: php:<tag>`.
- `composer.json#require.php` is a constraint, not an installed version. Ignore it for matching (mention in detail text only).

8.2.3 Python packages (MUST)

- Discover environments without executing Python. **Windows layouts (MUST):**
  - Project venvs: `<root>\**\.venv\Lib\site-packages`, plus `venv\` and `env\` (depth limit 4, skip `node_modules`). Note the capital `Lib` and no `python3.x` folder on Windows.
  - python.org installs: `%LOCALAPPDATA%\Programs\Python\Python3*\Lib\site-packages`.
  - User site: `%APPDATA%\Python\Python3*\site-packages`.
  - Microsoft Store Python: `%LOCALAPPDATA%\Packages\PythonSoftwareFoundation.Python.3.*\LocalCache\local-packages\Python3*\site-packages` (VERIFY on your machine).
  - Conda/Miniconda: `~\miniconda3\Lib\site-packages`, `~\miniconda3\envs\*\Lib\site-packages`, same for `anaconda3`.

  - pipx: `~\pipx\venvs\*\Lib\site-packages` and `%LOCALAPPDATA%\pipx\pipx\venvs\*\Lib\site-packages` (VERIFY which one your pipx uses).
  - uv tools: `%APPDATA%\uv\tools\*\Lib\site-packages` (VERIFY).
- macOS/Linux layouts (SHOULD): `.venv/lib/python3.*/site-packages`, `~/Library/Python/3.*/lib/python/site-packages`, `~/.local/lib/python3.*/site-packages`, `/opt/homebrew/lib/python3.*/site-packages`, `~/.local/pipx/venvs/*/lib/python3.*/site-packages`.
- Easiest robust approach: glob for `**/site-packages` and `**/Lib/site-packages` under each known base with a depth cap, then dedupe by real path (`fs.realpath`).
- Version from folder name: `litellm-1.107.0.dist-info` -> `1.107.0`. Read `METADATA` only if the folder name is ambiguous (it is a plain text file; reading it executes nothing).
- Also parse pins in `requirements*.txt`, `poetry.lock`, `uv.lock`, `Pipfile.lock`. A pin to `1.107.0` with no installed env is `likely` ("will install the bad version next time").

8.2.4 IOC files (MUST)

- In every discovered `site-packages`, `fs.stat` for `cook.pth`, `cook.py`, `sitecustomize.py`.
- For each hit: size + streamed SHA-256. Never read into memory as text, never print contents.
- `cook.pth` or `cook.py` present -> `definite` finding under `litellm-1.107.0` with action "Treat this environment as compromised: delete it, rotate keys it could read".
- `sitecustomize.py` alone -> `possible` (legit uses exist). Detail: "unexpected sitecustomize.py, inspect manually in a sandbox".
- VERIFY whether vendors published hashes; if so, add them to the pack and upgrade hash matches to `definite`.

8.2.5 npm lockfiles (SHOULD)

- `package-lock.json` (v2/v3 `packages` map), `pnpm-lock.yaml`, `yarn.lock` v1. Parse only; never install.
- No npm advisory in the pack is mandatory this week, but the parser feeds `kev-recent` product matching and makes the tool reusable next week. Keep it.

8.2.6 Env key names (MUST)

- Files: `.env`, `.env.*` (except `.env.example`, `.env.sample`, `.env.template`, which are reported separately as "template" and never trigger actions), `*.env`, `docker-compose*.yml` `environment:` keys.
- Algorithm (exact):

```
for await (const line of readLines(file)) {          // streaming
  const trimmed = line.trimStart();
  if (!trimmed || trimmed.startsWith("#")) continue;
  const eq = trimmed.indexOf("=");
  if (eq <= 0) continue;
  const key = trimmed.slice(0, eq).replace(/^export\s+/, "").trim();
  if (/^[A-Z_][A-Z0-9_]*$/i.test(key)) keys.push(key);
  // the right side is never sliced, never stored, never logged
}
```

- Provider map (for matching and for the "rotate" checklist): `RAILWAY_*` -> Railway; `OPENAI_API_KEY` -> OpenAI; `ANTHROPIC_API_KEY` -> Anthropic; `GOOGLE_API_KEY`, `GEMINI_API_KEY` -> Google; `AWS_*` -> AWS; `GITHUB_TOKEN`, `GH_TOKEN` -> GitHub; `DATABASE_URL` -> database.

8.2.7 Project and home markers (MUST)

- Railway: `railway.toml`, `railway.json` in project roots; `~\.railway` directory existence (do not read it); `railway` on PATH (on Windows check with `where.exe railway`, which also finds `.cmd` shims).
- OpenCode: `opencode` on PATH (`where.exe opencode`); config dir existence. VERIFY the Windows path; candidates are `~\.config\opencode`, `%APPDATA%\opencode`, `~\.local\share\opencode`. Existence only, never read.

8.2.8 Apps (MUST for Plex, SHOULD for GitLab)

- Plex Windows (MUST): check `%ProgramFiles%\Plex\Plex Media Server\Plex Media Server.exe` and `%ProgramFiles(x86)%\Plex\Plex Media Server\Plex Media Server.exe`. Read the version with `powershell.exe -NoProfile -Command "(Get-Item $env:TWOTOE_P).VersionInfo.ProductVersion"` where `TWOTOE_P` is the path passed as an env var. VERIFY that ProductVersion holds the full `1.43.x.yyyyy` build on your install.
- Plex macOS: `/Applications/Plex Media Server.app/Contents/Info.plist` -> `CFBundleShortVersionString` via `plutil -extract CFBundleShortVersionString raw <path>`. VERIFY the key holds the full `1.43.x.yyyyy` build; if not, try `CFBundleVersion`.
- Plex Linux: `dpkg-query -W -f='${Version}' plexmediaserver` or `rpm -q plexmediaserver`.
- Plex Docker: `plexinc/pms-docker:<tag>`. `latest` -> `possible` with "pull and recreate".
- GitLab: no native Windows install exists, so on Windows only Docker image tags count (`gitlab/gitlab-ce`, `gitlab/gitlab-ee`). On Linux also read `/opt/gitlab/embedded/service/gitlab-rails/VERSION`.

8.2.9 Trust stores (SHOULD)

- Windows root store (MUST on Windows): `powershell.exe -NoProfile -Command "Get-ChildItem Cert:\LocalMachine\Root, Cert:\CurrentUser\Root | Where-Object { $_.Subject -like ('*' + $env:TWOTOE_S + '*') } | Select-Object -ExpandProperty Subject"`. This matters because `curl.exe` on Windows uses Schannel, which trusts the Windows store. Microsoft blocked the roots on Oct 20, so presence alone is `possible`, not `likely`; VERIFY how the block appears (removal vs disallowed store) and check `Cert:\LocalMachine\Disallowed` too.
- Git for Windows ships its own CA bundle (VERIFY path, usually `C:\Program Files\Git\mingw64\etc\ssl\certs\ca-bundle.crt`). Text-search it; Git Bash `curl` uses it.
- Node: `tls.rootCertificates` of the user's Node (spawn `node -e` with a tiny script that prints subjects; this executes only Node core, not user code).
- Python certifi: glob `site-packages/certifi/cacert.pem` in every env; stream and search for the subject line text. No import.
- macOS system (SHOULD): `security find-certificate -a -c "emSign ECC Root CA - G3" /System/Library/Keychains/SystemRootCertificates.keychain` (presence is informational; Apple distrusts via trust settings, so presence alone is `possible`).
- curl on macOS (SHOULD): `/etc/ssl/cert.pem` text search.

8.2.10 Docker images (SHOULD)

- `docker images --format '{{.Repository}}:{{.Tag}}'` if `docker` exists and the daemon responds within 3 seconds. Docker Desktop on Windows is often not running at 3 a.m.; that is normal, skip silently and note "docker not running" in the brief meta.

8.3 Scan scope and performance budget

- Default roots on Windows: `~\code`, `~\projects`, `~\dev`, `~\Documents\GitHub`, `~\source\repos` (Visual Studio default), `~\Desktop`, if they exist, plus anything in config. Never scan `C:\`, `~`, or OneDrive roots recursively (OneDrive placeholders trigger downloads when read; skip files with the offline/recall-on-access attribute).
- Skip: `node_modules` (except for lockfile at the project root), `.git`, `dist`, `build`, `.next`, `AppData\Local\Temp`, `$Recycle.Bin`, `Library/Caches`.
- Max depth 6. Max files visited 200k. Target full scan < 40s on your laptop (Windows Defender real-time scanning slows filesystem walks; measure and report it). Record `durationMs` and `checksRun` (every detector invocation per target counts as one check).

9. Matcher

9.1 Rules

- Pure functions: `matchAll(inventory, pack, config) -> Finding[]`. No I/O. 100% unit-tested.
- One matcher file per advisory id in `match/matchers/`. Each exports `{ advisoryId, match(inventory, advisory, config): Finding[] }`.
- Dedupe by `(advisoryId, evidence location)`. Merge multiple evidence entries for the same root cause into one finding (for example three venvs with litellm 1.107.0 become one finding with three evidence rows, not three checklist items).
- `allowedTokens` = advisory CVEs + vendor + product + fixed versions + found versions + any numbers in `summary` + the evidence locations' basenames. This list is what the validator checks against.

9.2 Semver helpers (exact behavior)

```
// "Is the found version below the fixed version for its line?"
export function belowFixed(found: string, fixedByLine: Record<string, string>): "vulnerable" | "fixed"
    | "unknownLine" {
  const v = semver.coerce(found);            // tolerant of "8.3.35-cli"
  if (!v) return "unknownLine";
  const line2 = `${v.major}.${v.minor}`, line1 = `${v.major}`;
  const fixed = fixedByLine[line2] ?? fixedByLine[line1];
  if (!fixed) return "unknownLine";
  return semver.lt(v, semver.coerce(fixed)!) ? "vulnerable" : "fixed";
}
// Plex uses 4-part versions like 1.43.3.10861: compare as integer tuples, not semver.
export function cmpDotted(a: string, b: string): number { /* tuple compare, pad with 0 */ }
```

`unknownLine` for Node or PHP below the oldest supported line produces a `likely` finding: "Your Node 20.x line is out of support; upgrade to a patched LTS." (VERIFY Node 20 EOL status before shipping that string.)

9.3 Matcher truth table (tests MUST cover every row)

Input	Expected finding
node 24.22.0	nodejs `definite`, action upgrade to 24.22.1
node 24.22.1	none
node 26.12.0	nodejs `definite`
Dockerfile `FROM node:24-alpine`	nodejs `likely` (floating tag)
php 8.3.35	php2shell `definite`
php 8.3.36	none
php 8.1.30	php2shell `definite` + "take offline" wording
litellm 1.107.0 dist-info	litellm `definite`
litellm 1.106.2	none
`requirements.txt` has `litellm==1.107.0`, no env	litellm `likely`
`cook.pth` in site-packages	litellm `definite` (compromise wording)
only `sitecustomize.py`	litellm `possible`
`.env` with `RAILWAY_TOKEN` + `railway.toml`	railway `likely`

Input	Expected finding
`.env.example` with `RAILWAY_TOKEN` only	none (template file)
comment line `# RAILWAY_TOKEN=`	none
`opencode` on PATH	opencode `possible`
Plex 1.43.3.10860	plex2shell `definite`, KEV priority
Plex 1.43.3.10861	none
Node trust store contains emSign ECC Root CA - G3	emudhra `likely`
config.region = IN	sbi `fyi`

10. Writer, Prompts, Validator, Template

10.1 Ollama client (MUST)

```
POST http://127.0.0.1:11434/api/chat
{
  "model": process.env.TWOTOE_MODEL ?? "<gemma 4 e4b tag>",
  "stream": false,
  "format": <JSON schema generated from BriefDraft zod schema>,
  "options": { "temperature": 0.2, "seed": 42, "num_ctx": 8192 },
  "messages": [ { "role": "system", "content": SYSTEM }, { "role": "user", "content": USER } ]
}
```

- Timeout 90s per attempt (first call may load the model). `twotoe doctor` pre-warms with a tiny request.
- `BriefDraft` = `{ line1Actions: string[] (max 2), line2Tail: string, checklist: [{findingId, action, why}], fyi: string[] }`. Note the model does **not** write the counts. Code assembles `line1` and `line2` from the draft plus computed numbers.
- Record `prompt_eval_count`, `eval_count`, `total_duration` from the response for tracing and the post.

10.2 System prompt (MUST, store in `prompts.ts`, version it)

```
You are TwoToe, a calm morning security assistant. You write for a tired developer
who just woke up. You will receive a JSON list of FINDINGS that were already verified
by code. Your only job is to phrase them.

Rules:
1. Use ONLY facts present in FINDINGS. Never add a CVE ID, version number, vendor,
   product, date, or count that is not in that finding's allowedTokens.
2. One checklist item per finding id, in the given order. Never invent finding ids.
3. "action" is an imperative instruction, max 90 characters, starting with a verb.
4. "why" is one plain sentence, max 140 characters, no hype words
   (no: critical, urgent, devastating, catastrophic, broke, nightmare, actively,
    exploited) UNLESS the finding's severity is "critical" (then "critical" is allowed)
   or kev is true (then "actively exploited" is allowed).
5. For confidence "possible", start the action with "Confirm".
6. line1Actions: the two highest-priority actions, each max 60 characters, lowercase start.
7. line2Tail: max 60 characters, reassuring, no numbers.
8. fyi: rephrase each FYI finding in one calm sentence, max 160 characters.
9. Output JSON matching the schema. No markdown. No extra keys.
```

10.3 User prompt template

```
DATE: {{date}}
FINDINGS (ordered by priority, already verified):
{{findings_json}}

FYI (awareness only, never actions):
{{fyi_json}}
```

10.4 Few-shot example (SHOULD, include one in the system prompt)

Input: one `definite` Node finding (24.22.0 -> 24.22.1) and one `possible` OpenCode finding. Output:

```
{
  "line1Actions": ["bump Node to 24.22.1"],
  "line2Tail": "go back to sleep",
  "checklist": [
    { "findingId": "nodejs-2026-10-21:a1", "action": "Upgrade Node from 24.22.0 to 24.22.1",
      "why": "The Oct 21 Node release fixes a Brotli decoder bug and a TLS hostname check bypass." },
    { "findingId": "opencode-console-2026-10-17:b2", "action": "Confirm whether you linked API keys in
        OpenCode Console",
      "why": "OpenCode disclosed that Console provider keys were exposed for some users; revoke them if
          you linked any." }
  ],
  "fyi": []
}
```

Note: line 1 only lists the definite/likely action. The possible item stays in the checklist.

10.5 Grounding validator (MUST, the star of the post)

```
export function validate(draft: BriefDraft, findings: Finding[]): string[] /* rejection reasons */ {
  const reasons: string[] = [];
  const byId = new Map(findings.map(f => [f.id, f]));
  // 1. ids
  for (const item of draft.checklist)
    if (!byId.has(item.findingId)) reasons.push(`unknown findingId ${item.findingId}`);
  if (new Set(draft.checklist.map(i => i.findingId)).size !== draft.checklist.length)
    reasons.push("duplicate findingId");
  // 2. coverage: every definite/likely/possible finding appears exactly once
  // 3. per-item grounding
  for (const item of draft.checklist) {
    const f = byId.get(item.findingId); if (!f) continue;
    const allowed = new Set(f.allowedTokens.map(norm));
    for (const tok of extractRiskyTokens(item.action + " " + item.why))
      if (!allowed.has(norm(tok))) reasons.push(`ungrounded token "${tok}" in ${f.id}`);
    // 4. hype words gated by severity/kev
    // 5. "possible" must start with "Confirm"
    // 6. length caps (zod already enforces; re-check after trim)
  }
  // 7. line1Actions tokens must be a subset of the union of top-2 findings' allowedTokens
  // 8. line2Tail must contain no digits
  return reasons;
}
```

`extractRiskyTokens` pulls:

- CVE IDs: `/CVE-\d{4}-\d{4,}/gi`
- Version-like numbers: `/\b\d+(?:\.\d+){1,3}\b/g`
- Bare numbers >= 2 digits: `/\b\d{2,}\b/g` (years like 2026 must be in allowedTokens if used)
- Capitalized product/vendor names from a known list (Node, PHP, Plex, GitLab, LiteLLM, Railway, OpenCode, OpenAI, Anthropic, Google, eMudhra, SBI, CISA) plus any capitalized token that matches a known-vendors list but is not allowed for that finding.

Retry policy: attempt 1 -> validate -> if rejected, attempt 2 with the rejection reasons appended to the user prompt ("Your previous draft was rejected for: ..."). If attempt 2 fails, use `template.ts` for the failing items only (mode `llm-fallback`), keep valid items. Record every rejection reason in `brief.meta.validatorRejections`. Those reasons become the "What broke" section of the post.

10.6 Template writer (MUST)

Pure function, no model. For each finding: `action = advisory.action` with `{found}` and `{fixed}` substituted; `why = advisory.summary` truncated to 140 at a word boundary. `line1Actions` = first two `definite|likely` actions shortened by a per-advisory `shortAction` field (add it to the pack). This is both the fallback and the eval baseline ("template vs model: which reads better, and what does the model cost in latency?").

10.7 Line assembly (code, not model)

```
const actionable = findings.filter(f => f.confidence === "definite" || f.confidence === "likely");
line1 = actionable.length === 0
  ? "Nothing needs you today."
  : `${actionable.length} thing${actionable.length > 1 ? "s" : ""} need${actionable.length > 1 ? "" :
      "s"} you today: `
    + draft.line1Actions.slice(0, 2).join("; ")
    + (actionable.length > 2 ? `; +${actionable.length - 2} more in the checklist.` : ".");
line2 = `Everything else: you're fine. Checked ${inv.checksRun} things in ${secs}s, 0 secrets left this
    machine.`;
```

The "0 secrets left this machine" claim is computed: it is only printed if the egress ledger has zero requests to non-local hosts other than the KEV feed (which carries no user data). Otherwise line 2 says exactly what left and where.

11. Partner Track Integrations

11.1 Strategy

You can only win once per challenge, and DEV wants to spread wins. So partner tech is not about collecting categories, it is about making the project better in ways a judge can see. Each integration below is rated by **fit** (does it make TwoToe genuinely better?) and **cost** (hours). Claim a category in the post only if the integration is load-bearing and visible in the demo.

Partner / category	Prize	Fit	Hours	Decision
Gemma (featured)	$200	Core: the local writer model	0 extra	**MUST**
Temporal	$100	Core: durable nightly workflow + schedule + crash test	4	**MUST**
Sentry Agent Tracing	$100	Strong: traces every model call, validator rejection, retry	3	**SHOULD**
.tech Domain	$100	Light but free: `twotoe.tech` for demo + install script	0.5	**SHOULD**
Render or **DigitalOcean** (featured)	$200	Hosts the static demo card. Pick ONE (11.6)	1	**SHOULD**
Mastra	$100	Stretch: "Ask TwoToe" grounded follow-up agent with tools	4	**STRETCH**
ElevenLabs	$100	Stretch: opt-in voice brief. Weakens privacy story	2	**STRETCH, opt-in**
Arduino (featured)	$200	Fun stretch: a desk sloth LED that glows if Line 1 > 0	3 + hardware	**STRETCH, only if you own a board**
Tinker (featured)	$200	Could fine-tune the writer, but you won Tinker in Week 1	6	**SKIP** (repeat-winner headwind)
TabPFN (featured)	$200	No tabular prediction in this product	n/a	SKIP
MongoDB Atlas, Tiger Data	$100	Cloud DB contradicts "nothing leaves the machine"	n/a	SKIP, and say why in the post
Browserbase, SerpApi	$100	Cloud fetching of advisories; pack is curated and local	n/a	SKIP
Auth0 for AI Agents, Backboard, Entire, Ultramarine, GitHub Copilot	$100	No genuine fit for a local CLI	n/a	SKIP

Before claiming any category: re-read that category's exact text on the challenge page (dev.to/challenges/hacktoberfest-week3-2026-10-19). Category requirements can be specific (for example a particular product or API). Adjust the integration to match, or drop the claim.

11.2 Gemma (MUST)

- Default model: Gemma 4 E4B via Ollama. VERIFY the exact tag on ollama.com/library.
- Make the model a first-class, visible part of the post: name it in the title or subtitle, show tokens/sec and latency on your Windows laptop (with its specs), show the validator's rejection rate for Gemma.

- **SHOULD: model bake-off in the eval.** Run the writer with 2 or 3 local models (for example Gemma 4 E2B, Gemma 4 E4B, and one non-Gemma 4B) on the same 24 fixtures. Report unsupported-claim rate, fallback rate, median latency. Your Gemma 4 variants post already gives you credibility here; link it.
- Write the "why Gemma" paragraph honestly: small enough to run on a laptop overnight, good at following a JSON schema, and its mistakes are catchable by the validator.

11.3 Temporal (MUST)

Workflow

```
// packages/worker/src/workflows.ts
import { proxyActivities } from "@temporalio/workflow";
import type * as acts from "./activities";

const fast = proxyActivities<typeof acts>({
  startToCloseTimeout: "2 minutes",
  retry: { maximumAttempts: 3, initialInterval: "5s", backoffCoefficient: 2 },
});
const model = proxyActivities<typeof acts>({
  startToCloseTimeout: "5 minutes",
  heartbeatTimeout: "30 seconds",
  retry: { maximumAttempts: 4, initialInterval: "10s", backoffCoefficient: 2,
           nonRetryableErrorTypes: ["ValidationExhausted"] },
});

export async function nightlyTriage(input: { configPath: string; offline: boolean }) {
  const inv = await fast.collectInventory(input.configPath);
  const pack = input.offline ? await fast.loadPack() : await fast.refreshKevAndLoadPack();
  const findings = await fast.matchAdvisories(inv, pack);
  const draft = await model.writeBriefWithValidation(findings);   // includes retry + fallback
  const brief = await fast.assembleBrief(inv, findings, draft);
  await fast.persistAndNotify(brief);
  return { date: brief.date, actions: brief.checklist.length, mode: brief.meta.mode };
}
```

- Activities are idempotent: each caches its output on disk keyed by a hash of its input under `~/.twotoe/cache/<workflowId>/`. A retried `writeBriefWithValidation` after a crash reuses a finished model response if one exists, so "no result is computed twice" (your Week 1 line, now proven again).
- `writeBriefWithValidation` heartbeats every 10s while waiting on Ollama.

Schedule

```
// packages/worker/src/schedule.ts
await client.schedule.create({
  scheduleId: "twotoe-nightly",
  spec: { calendars: [{ hour: 3, minute: 0 }] },      // local time; set timezone via spec.timezone
      (VERIFY field name)
  action: { type: "startWorkflow", workflowType: "nightlyTriage",
            taskQueue: "twotoe", args: [{ configPath, offline: false }] },
  policies: { overlap: "SKIP", catchupWindow: "6h" }, // laptop was asleep at 3 a.m.? catch up on wake
});
```

The `catchupWindow` is the laziness feature that is also a Temporal feature: if your laptop was asleep at 3 a.m., the run happens when it wakes, before you do. Say that in the post.

Running it on Windows

- `twotoe sleep` checks for a Temporal dev server on `127.0.0.1:7233`; if missing, prints the install hint from 5.4 and `temporal server start-dev --db-filename %USERPROFILE%\.twotoe\temporal.db`.
- `twotoe sleep --taskscheduler` (SHOULD) registers two Windows scheduled tasks via PowerShell `Register-ScheduledTask`, both triggered "At log on" for the current user, running hidden: one starts the Temporal dev server, one starts the worker. This survives reboots without admin rights.

- Fallback `twotoe sleep --no-temporal` registers one daily task at 03:00 that runs `twotoe brief`, with `New-ScheduledTaskSettingsSet -WakeToRun -StartWhenAvailable`. `StartWhenAvailable` is Task Scheduler's version of `catchupWindow`, and `WakeToRun` will actually wake a sleeping laptop if your power plan allows wake timers. Mention both in the post.
- Hidden windows: launch through `powershell.exe -NoProfile -WindowStyle Hidden -Command ...` or `conhost.exe --headless` (VERIFY availability) so no console flashes at 3 a.m.
- `twotoe sleep --uninstall` removes every task it created (names prefixed `TwoToe-`).
- macOS (SHOULD, for readers): `--launchd` installs equivalent LaunchAgents.

Crash test (MUST, it is a demo moment)

`packages/worker/scripts/crash-test.ts` (run with `pnpm tsx`, works on Windows):

1. Start dev server and worker.
2. Start `nightlyTriage` on the fixture config with an injected 20s delay inside the model activity.
3. After 8s, hard-kill the worker: on Windows `taskkill /PID <pid> /T /F` (the `/T` kills the child tree), elsewhere `process.kill(pid, "SIGKILL")`.
4. Restart the worker.
5. Query the workflow history with `temporal workflow show` and print: activities completed before the crash, activities retried, activities executed twice (must be 0).
6. Screen-record the Temporal UI timeline at `localhost:8233`. That recording goes in the post.

11.4 Sentry Agent Tracing (SHOULD)

Goal: one trace per nightly run that a judge can read like a story: scan -> match -> model attempt 1 (rejected, reason) -> model attempt 2 (accepted) -> persist.

- VERIFY the current Sentry AI agent monitoring docs for Node (manual instrumentation) before writing code. Use the documented span ops and `gen_ai.*` attributes. The shape is roughly:

```
await Sentry.startSpan({ op: "gen_ai.invoke_agent", name: "invoke_agent TwoToe",
  attributes: { "gen_ai.agent.name": "TwoToe", "gen_ai.request.model": model } }, async () => {
  for (const attempt of [1, 2]) {
    await Sentry.startSpan({ op: "gen_ai.chat", name: `chat ${model}`,
      attributes: { "gen_ai.request.model": model, "gen_ai.request.temperature": 0.2 } }, async (span)
          => {
      const res = await ollamaChat(...);
      span.setAttribute("gen_ai.usage.input_tokens", res.prompt_eval_count);
      span.setAttribute("gen_ai.usage.output_tokens", res.eval_count);
    });
    await Sentry.startSpan({ op: "gen_ai.execute_tool", name: "execute_tool groundingValidator" },
        async (span) => {
      span.setAttribute("twotoe.validator.rejections", reasons.length);
    });
  }
});
```

- Privacy: `sendDefaultPii: false`. Prompt and response content attributes only when `TWOTOE_TRACE_CONTENT=demo` and the run uses fixtures. Add a `beforeSendTransaction` hook that strips any attribute whose value contains `C:\Users\`, `/Users/`, or `/home/`, or matches a key-name regex, as a belt-and-braces guard.
- The Sentry ingest host is added to the egress allowlist only when `SENTRY_DSN` is set and `TWOTOE_TELEMETRY=1`. The card's egress ledger will show it. This is honest and is itself a nice detail in the post ("the only thing that leaves is a trace with token counts, and you can turn it off").
- Post asset: a screenshot of the trace waterfall with a visible rejected attempt.

11.5 .tech domain (SHOULD)

- Claim the free domain via hacktoberfest.com/my/promos. Suggested: `twotoe.tech` (fallbacks: `twotoe-sloth.tech`, `sleepytriage.tech`).
- Point it at the demo card host. Serve the README-style landing at `/` with the example brief and an install snippet.
- Do NOT host a `curl | sh` installer. Security tool, security story: tell users to clone and `pnpm i`. Say why in one line.

11.6 Render or DigitalOcean (SHOULD, pick one)

The card is a static Vite build. Either host works in 15 minutes. Decide by reading both featured category descriptions on the challenge page:

- If the DigitalOcean category is about deploying anything on DO, use **App Platform static site** and claim it (featured, $200).
- If the DigitalOcean category specifically requires its AI or GPU products, do not force it. Use **Render static site** (credits via promos) and claim Render only if its category text fits a static deploy.
- Either way: build with `pnpm --filter card build`, publish `apps/card/dist`, set the custom `.tech` domain, enable HTTPS.

11.7 Mastra "Ask TwoToe" (STRETCH)

A follow-up Q&A agent: `twotoe ask "why does the Node one matter?"`.

- Mastra agent with an Ollama-backed model (via the AI SDK Ollama provider; VERIFY the current provider package).
- Tools: `getFinding(id)`, `listFindings()`, `getAdvisory(id)`. No filesystem tool, no web tool.
- Answers are run through the same validator (allowed tokens = union of tools' returned tokens).
- Only worth it if every MUST and SHOULD is green by Oct 24 evening IST.

11.8 ElevenLabs voice brief (STRETCH, opt-in)

- Default voice output is Windows' built-in speech engine (fully local): `powershell.exe -NoProfile -Command "Add-Type -AssemblyName System.Speech; (New-Object System.Speech.Synthesis.SpeechSynthesizer).Speak($env:TWOTOE_TEXT)"` with the text passed via env var. macOS fallback: `say`.
- `--voice elevenlabs` sends **only the two lines** (which contain no paths, no key names, only generic actions) to ElevenLabs TTS. Show the egress ledger entry for it. Opt-in, off by default, documented in the post as a deliberate trade-off.

11.9 Arduino desk sloth (STRETCH, hardware only)

If you own an Arduino with Wi-Fi or USB serial: the worker writes the action count to the serial port at 7 a.m.; the board fades a LED green (0), amber (possible only), red (1+ definite/likely). A 5-second clip of the LED turning red on your desk is a great GIF. Skip entirely if you would need to buy hardware.

12. CLI Spec

Command	Purpose	Key flags	Exit code
`twotoe doctor`	Check Windows specifics (PATH shims, execution policy, long paths), Node version (and warn if TwoToe itself runs on a vulnerable Node), Ollama reachable, model present, Temporal reachable, config valid. Pre-warm model	`--fix` prints install commands	0 ok, 2 problem
`twotoe scan [roots...]`	Inventory only	`--json`, `--fixture <dir>`	0
`twotoe brief`	Scan + match + write + print the two lines + checklist	`--no-llm`, `--offline`, `--model`, `--fixture`, `--json`, `--open`	0 nothing, 10 actions, 2 error
`twotoe sleep`	Install nightly schedule	`--at 03:00`, `--taskscheduler`, `--no-temporal`, `--uninstall`	0
`twotoe card`	Serve the latest brief on `127.0.0.1:4777` and open it	`--port`	0
`twotoe explain <findingId>`	Show evidence rows, advisory, sources		0
`twotoe pack`	Show pack version, verifiedAt dates, refresh KEV	`--refresh`	0
`twotoe eval`	Run eval harness, write `eval/RESULTS.md`	`--models a,b,c`, `--runs 5`	0
`twotoe ask "<q>"`	STRETCH Mastra agent		0

Terminal output of `twotoe brief` (this exact look goes in the GIF):

```
  twotoe  Thu Oct 23, 07:00

  2 things need you today: roll your Railway env vars; bump Node to 24.22.1.
  Everything else: you're fine. Checked 412 things in 14.2s, 0 secrets left this machine.

  [ ] 1  Roll every env var stored in Railway            likely    railway.toml, .env (RAILWAY_TOKEN)
  [ ] 2  Upgrade Node from 24.22.0 to 24.22.1            definite  node --version
  [ ] 3  Confirm whether you linked keys in OpenCode...  possible  opencode on PATH

  FYI  SBI disclosed a customer data exposure; expect phishing, SBI never asks for OTP or PIN.

  card: http://127.0.0.1:4777   model: gemma4 e4b   validator: 1 rejection, fixed on retry
```

Config at `~/.twotoe/config.json`:

```
{
  "roots": ["~/code", "~/Documents/GitHub", "~/source/repos"],
  "region": "IN",
  "providers": { "railway": "auto", "opencodeConsole": "unknown" },
  "model": "<gemma tag>",
  "notify": { "toast": true, "voice": "off" },
  "telemetry": { "sentry": false }
}
```

13. The Morning Card (apps/card)

13.1 Design direction

Calm, warm, sleepy. Not a SOC dashboard. Think "a note left on the kitchen counter". Use your Tailwind skills, but restraint is the aesthetic.

- Layout: single column, max-width 640px, centered. Big two lines at the top in a serif display face (for example Fraunces or Instrument Serif via Google Fonts with a system serif fallback) on a soft paper background; everything else in a clean sans.
- Color tokens: paper `#F7F3EA`, ink `#1F1D1A`, moss (fine) `#5B7F5A`, amber (possible) `#C98A2B`, clay (action) `#B5523B`. Dark mode: night `#16161A`, ink `#ECE7DC`, same accents slightly lifted.
- An original, simple sloth illustration (inline SVG you draw yourself: rounded body, two toes, closed eyes). Not a known mascot.

13.2 Components

Component	Content
`Header`	Date, "good morning" line, model + mode badge (llm / template / llm-fallback)
`TwoLines`	Line 1 in clay if actions > 0, moss if none. Line 2 in muted ink
`Checklist`	Items with checkbox (state in `localStorage`, keyed by brief date + findingId), action, why, confidence chip, expandable evidence rows, "Vendor advisory" link
`Fyi`	Up to 3 lines, muted
`EgressLedger`	Table of every outbound request: host, method, bytes out. Highlights "0 secrets left this machine"
`RunStats`	Checks run, scan time, model latency, validator rejections (expandable list of reasons)
`ExampleSwitcher`	Demo only: "See an Example" buttons for 3 bundled fixture briefs: clean night, Railway user, LiteLLM compromise
`Footer`	"TwoToe can be wrong. Every item links to the vendor." + repo link

13.3 Data loading

- Local mode: `twotoe card` serves `/brief.json` from `~/.twotoe/briefs/latest.json`.
- Demo mode (hosted): bundled `example/*.json`, no network calls at all. The hosted page must work with JavaScript fetching only same-origin static files.

13.4 Acceptance

- Lighthouse accessibility >= 95. Keyboard navigable checklist. Respects `prefers-reduced-motion` and `prefers-color-scheme`.
- Works at 360px width (judges open links on phones).

14. Eval Harness (the numbers that win Writing Quality)

14.1 What you will claim in the post

Only numbers you actually measured. Target claims (fill with real results):

- Matcher precision and recall on 24 synthetic fixtures (target 100% / 100%; if not, show the miss).
- Writer unsupported-claim rate **before** the validator vs **after** (for example "Gemma invented a version number in 7 of 120 drafts; the validator caught all 7; 3 needed the template fallback").
- Model bake-off: 2 or 3 local models, same fixtures.
- Median end-to-end time on your Windows laptop (scan + model + validate), with CPU/GPU/RAM listed.
- The lazy payoff: your stopwatch time to manually triage the 10-item viral post against your own machine vs TwoToe's time.
- Egress: bytes of user data sent off-machine = 0 (ledger proof).

14.2 Fixtures (MUST, generated by `eval/gen-fixtures.ts`)

Each fixture dir has `probe.json` (machine facts for `FixtureProbe`), `project/` (synthetic files), and `expected.json` (ground-truth findings: advisoryId + confidence).

#	Name	Seeded condition	Expected
f01	litellm-bad-venv	`.venv/.../litellm-1.107.0.dist-info`	litellm definite
f02	litellm-ioc	`cook.pth` + `cook.py` in site-packages	litellm definite (compromise)
f03	litellm-pin-only	`requirements.txt` pins 1.107.0, no venv	litellm likely
f04	litellm-safe	1.106.2 installed	none
f05	sitecustomize-only	legit-looking `sitecustomize.py`	litellm possible
f06	railway-user	`railway.toml` + `.env` with `RAILWAY_TOKEN`	railway likely
f07	railway-example-only	`.env.example` with `RAILWAY_TOKEN`	none
f08	railway-comment	`# RAILWAY_TOKEN=` commented	none
f09	opencode-cli	probe: `opencode` on PATH	opencode possible
f10	node-old-24	probe node 24.22.0	nodejs definite
f11	node-patched	probe node 24.22.1	none
f12	node-docker-floating	`FROM node:24-alpine`	nodejs likely
f13	node-20	probe node 20.x	nodejs likely (unsupported) VERIFY wording
f14	php-83-old	probe php 8.3.35	php2shell definite
f15	php-81	probe php 8.1.30	php2shell definite (take offline)
f16	php-patched	probe php 8.4.27	none
f17	plex-old	probe plex 1.43.3.10860	plex2shell definite, KEV

#	Name	Seeded condition	Expected
f18	plex-patched	probe plex 1.43.3.10861	none
f19	gitlab-docker	`gitlab/gitlab-ce:<old tag>`	gitlab definite (VERIFY tag)
f20	emudhra-node-store	probe node trust store contains subject	emudhra likely
f21	region-in	config region IN, nothing else	sbi fyi only
f22	kitchen-sink	f01 + f06 + f10 + f17	4 findings, correct order
f23	clean-machine	everything patched	"Nothing needs you today."
f24	clean-tricky	near-misses: 24.22.1, 8.3.36, 1.106.2, `.env.sample`, `CHANGELOG` mentioning CVEs	none

Canary rule: every fixture `.env` contains `SECRET_VALUE=TWOTOE_CANARY_9f3a`. `run-eval.ts` greps every output artifact (briefs, logs, traces exported to disk, cache) for the canary. Any hit fails the whole eval.

14.3 Metrics and outputs

`twotoe eval --models <a>,<b> --runs 5` produces `eval/RESULTS.md` with these tables (paste-ready for the post):

1. **Matcher**: per advisory TP / FP / FN, overall precision, recall.
2. **Writer per model**: drafts, drafts with any ungrounded token (before validator), rejection reasons histogram, retries, fallbacks, final ungrounded tokens after validator (must be 0), median and p90 latency, tokens/sec.
3. **End-to-end**: median total time per fixture.
4. **Canary**: hits (must be 0).
5. A sample of 3 rejected drafts with the offending token highlighted (raw material for "What broke").

14.4 Human readability check (SHOULD)

Ask two people (Subarna counts, if willing) to rank template vs model briefs blind for 6 fixtures. Report the result whatever it is. If the template wins, that is a great honest paragraph.

15. Testing Strategy

Layer	Tests (Vitest)	Gate
schemas	round-trip valid/invalid samples	Phase 0
semver utils	every truth-table row in 9.3 plus 4-part Plex compare	Phase 2
env scanner	canary never appears; `export KEY=`; quoted values; comments; CRLF files; EBUSY/EPERM skip; 10 MB file streams without memory spike	Phase 1
IOC scanner	never opens file for text read (spy on `fs.readFile`); hash correct	Phase 1
python env discovery	dist-info parsing, METADATA fallback, depth limit, symlink loops	Phase 1
matcher	one test per truth-table row; dedupe/merge behavior	Phase 2
validator	unknown id, duplicate id, invented CVE, invented version, hype word gating, Confirm rule, digits in line2Tail, coverage	Phase 3
writer	mocked Ollama: valid draft, invalid-then-valid, invalid-twice -> fallback	Phase 3
egress guard	blocked host throws; ledger records; offline mode makes zero non-local requests	Phase 3
workflow	Temporal `TestWorkflowEnvironment` happy path + activity failure retry	Phase 4
card	component render with each example brief; checkbox persistence	Phase 5
e2e	`twotoe brief --fixture eval/fixtures/f22-kitchen-sink --no-llm` snapshot	Phase 6

Coverage target: 90% lines in `core` and `writer`. CI: GitHub Actions matrix `windows-latest` (primary) and `ubuntu-latest`, running `pnpm -r test` on Node 24.22.1 (no Ollama in CI; writer tests use the mock). A green Windows badge in the README is a nice touch for the post.

15.1 Egress guard implementation note

```
// packages/core/src/egress/guard.ts
const ALLOW = new Set(["127.0.0.1", "localhost", "www.cisa.gov"]);
export function installEgressGuard(opts: { extraHosts?: string[]; ledger: Ledger }) {
  for (const h of opts.extraHosts ?? []) ALLOW.add(h);
  const realFetch = globalThis.fetch;
  globalThis.fetch = async (input, init) => {
    const url = new URL(typeof input === "string" ? input : input instanceof URL ? input.href :
        input.url);
    if (!ALLOW.has(url.hostname)) throw new EgressBlockedError(url.hostname);
    const bytesOut = init?.body ? Buffer.byteLength(String(init.body)) : 0;
    opts.ledger.record({ host: url.hostname, method: init?.method ?? "GET", bytesOut });
    return realFetch(input, init);
  };
}
```

Also guard `http`/`https` module requests (Sentry SDK may use them) by checking at the transport level, or configure Sentry's transport to use the guarded fetch. Temporal gRPC goes to localhost only; assert the client address is loopback at startup.

16. Phased Build Plan (with Claude Code prompts and gates)

Times are focused-work estimates. Total MUST + SHOULD is roughly 35 to 40 hours. You have from Oct 22 evening to Oct 25 evening IST. That is tight but doable if you protect Phase 8 (the post).

Phase	Name	Est.	Target finish (IST)
0	Repo, schemas, pack	2h	Oct 22 night
1	Probe + scanners	6h	Oct 23 midday
2	Matcher + `scan`/`brief --no-llm`	3h	Oct 23 afternoon
3	Writer + validator + egress guard	5h	Oct 23 night
4	Temporal workflow, schedule, crash test	4h	Oct 24 morning
5	Card UI + deploy + domain	5h	Oct 24 afternoon
6	Fixtures + eval + RESULTS.md	4h	Oct 24 night
7	Sentry tracing	3h	Oct 25 morning
8	Post, GIF, video, real-machine run	6h	Oct 25 evening
9	Stretch (only if all green)	any	never past Oct 25 noon
Freeze	Tag `v1.0-submission`, publish	1h	**Oct 25, 9 PM IST at the latest**

Phase 0: Scaffold (MUST)

Claude Code prompt:

```
Read docs/SPEC.md sections 0 to 7 and CLAUDE.md. Then:
1. Scaffold the pnpm monorepo exactly as in 5.1 (empty files are fine for later phases).
2. Pin exact dependency versions (run `pnpm view <pkg> version`), strict TS, Vitest.
3. Implement packages/core/src/schemas.ts exactly as in section 6, with tests.
4. Create packages/core/src/pack/advisories.json from section 7.2 and 7.3.
   For every VERIFY field, fetch the source URL, confirm the value, and set verifiedAt.
   If you cannot confirm a value, leave it as "VERIFY" and list it in docs/VERIFY.md.
5. Add README skeleton with the "built during Hacktoberfest Week 3" note.
Stop and show me: tree, test output, and docs/VERIFY.md.
```

Gate 0: tests green, pack validates against the zod schema, VERIFY list reviewed by you.

Phase 1: Probe and scanners (MUST)

Claude Code prompt:

```
Read SPEC 5.4 (Windows rules) first. Implement section 8 fully: SystemProbe interface,
RealProbe (Windows first; macOS/Linux branches can be stubs for now; execFile or
cross-spawn with arg arrays only, 5s timeouts), FixtureProbe, and every scanner in 8.2. Follow section
    3 hard rules exactly:
never read env values, never open IOC files as text, never import Python.
Write the tests listed for scanners in section 15, including the canary test.
Then run `pnpm tsx packages/cli/src/index.ts scan $HOME\code --json | Select-Object -First 50`
in PowerShell on my machine and summarize what it found, with every path redacted to ~.
Stop for review.
```

Gate 1: canary test green, IOC test proves no text read, real scan under 20s, output contains no values.

Phase 2: Matcher (MUST)

Claude Code prompt:

```
Implement section 9: semver utils, one matcher per advisory, matchAll, priority (7.4),
allowedTokens construction. Write one test per row of the 9.3 truth table.
Wire `twotoe brief --no-llm` using the template writer from 10.6 and line assembly from 10.7.
Show me the output for a temporary fixture that combines f01, f06, f10, f17.
```

Gate 2: truth table 100% green; `brief --no-llm` prints the exact layout from section 12.

Phase 3: Writer, validator, egress (MUST)

Claude Code prompt:

```
Implement section 10 (Ollama client with JSON-schema format, prompts verbatim, validator,
retry-with-reasons, partial template fallback) and the egress guard from 15.1.
Every model call goes through the guard. Write all validator tests from section 15.
Then run `twotoe brief --fixture <kitchen-sink>` 5 times with the real local model and
report: rejections per run, reasons, final mode, latency. Do not hide failures.
```

Gate 3: validator tests green; 5 real runs end with zero ungrounded tokens; egress ledger shows only localhost.

Phase 4: Temporal (MUST)

Claude Code prompt:

```
Implement section 11.3: workflow, activities (idempotent with on-disk cache keyed by input
hash), worker, schedule with catchupWindow, `twotoe sleep` (+ --taskscheduler, --no-temporal),
and scripts/crash-test.ts (taskkill /T /F on Windows). Use TestWorkflowEnvironment for tests. Run the
    crash test and
paste the summary (completed before crash, retried, executed twice = must be 0).
```

Gate 4: crash test proves 0 duplicate executions; schedule visible in Temporal UI.

Phase 5: Card + deploy (SHOULD)

Claude Code prompt:

```
Read the frontend design guidance you have available, then build apps/card per section 13.
Original inline SVG sloth, paper/ink palette, dark mode, 360px support, a11y >= 95.
Bundle three example briefs generated from fixtures f22, f06, f23 (no real paths).
Add `twotoe card` local server. Prepare deploy config for <Render|DigitalOcean> static
site and document custom domain steps for <twotoe.tech>.
```

Gate 5: hosted demo live on the `.tech` domain, "See an Example" works on a phone.

Phase 6: Eval (MUST)

Claude Code prompt:

```
Implement section 14: gen-fixtures.ts creating all 24 fixtures with probe.json, project/,
expected.json, and the canary; run-eval.ts producing every table in 14.3 into eval/RESULTS.md.
Run it with models: <gemma e4b tag>, <gemma e2b tag>, <one non-Gemma ~4B> and --runs 5.
Report results honestly, including any matcher misses, and propose fixes but do not
change expected.json to make numbers look better.
```

Gate 6: canary hits 0; matcher misses explained or fixed in code (never by editing ground truth).

Phase 7: Sentry (SHOULD)

Claude Code prompt:

```
Fetch and read the current Sentry docs for AI agent monitoring in Node (manual
instrumentation). Implement section 11.4 with the documented span ops and attributes,
opt-in only, content only in demo mode, plus the scrub hook. Add the Sentry ingest host
to the egress allowlist only when telemetry is enabled. Run one demo trace on the
kitchen-sink fixture with a forced first-attempt rejection and give me the trace link.
```

Gate 7: one clean trace with a visible rejected attempt; ledger shows the Sentry host only when enabled.

Phase 8: Ship the story (MUST)

- Run TwoToe on your real Windows machine. Note aggregate counts only.
- Stopwatch the manual triage baseline before you look at TwoToe's output (honesty).
- Record: terminal GIF (asciinema + agg, or a screen recording converted to GIF), Temporal crash-test clip, card on phone, Sentry trace screenshot.
- Write the post from Section 17. Have Claude Code proofread for facts only (every number must trace to RESULTS.md or a source link), not for voice.
- Tag `v1.0-submission`, push, publish.

Phase 9: Stretch (only if everything above is green)

Priority order: model bake-off extras -> Mastra ask -> Arduino LED (if hardware) -> ElevenLabs opt-in.

17. The DEV Post (draft skeleton)

Voice: yours. Casual, specific, a little cheeky, numbers first. Replace every `{{...}}` with a measured value from `eval/RESULTS.md`. If a number is not measured, cut the sentence.

17.1 Title options (pick after the eval)

- "Breach week dropped 10 'patch NOW' posts. A local Gemma read them while I slept and gave me 2 lines."
- "I refused to triage breach week. So I built a sloth that does it overnight, on my laptop."
- "8 of 10 viral breach claims were real. My 4B model found the {{k}} that hit my machine before breakfast."

Cover image: the card screenshot (example brief) with the sloth, 1000x420.

17.2 Body skeleton

Hook (3 short paragraphs). The viral post. "8 of these 10 are real. One number is off by about 2,000x. And I was not about to paste my `.env` into a chatbot to find out which ones hit me, because half of this week's incidents were literally about leaked keys."

TL;DR box.

- What: a local agent that does breach-week triage overnight and hands you two lines at 7 a.m.
- Model: Gemma 4 E4B on my Windows laptop via Ollama, {{tokens/s}} tok/s ({{CPU or GPU}}).
- Proof: {{precision}}/{{recall}} matcher on 24 fixtures; validator caught {{n}} invented facts; 0 ungrounded tokens shipped.
- Privacy: reads key names, never values; egress ledger shows 0 bytes of user data left the machine.
- Demo: {{twotoe.tech}} | Repo: {{github.com/SoumyaEXE/twotoe}}

What I Built. Who gets lazier (solo devs, students, freelancers). The two-line contract. One screenshot.

Demo. GIF of `twotoe brief`, link to the hosted card with "See an Example".

The fact-check (condensed). A 10-row table: claim, verdict, one-line reality. Highlight: eMudhra 60M vs about 30,000 needing migration; OpenCode was Oct 17 not "today"; "AI is exploiting them" has no source. Rajasthan Police: unverified, so it is not in my pack. This shows judgment, and judgment is Writing Quality.

How I Built It. The two-walls diagram (matcher -> model -> validator). Short code snippets: env scanner loop (the "right side is never stored" comment), the validator token check, the Temporal schedule with `catchupWindow`.

What Broke (the honesty section). 3 real rejected drafts with the invented token highlighted, the rejection reason, and what the retry or template produced. Any matcher miss and how you fixed it in code. If the template beat the model in the blind read, say so.

Why Does Open Innovation Matter? "Railway, OpenCode, and LiteLLM were all credential incidents. A triage tool that needs your credentials in someone else's cloud is just incident number eleven." Local model, local facts, works offline, works when your cloud keys are revoked mid-incident, costs $0 per night forever, swap the model with one env var.

My Agent Session. Embed or link the Claude Code session per the template's instructions. Pick the moment where the validator spec caught Claude Code (or the model) inventing something; that is a great narrative beat.

The Lazy Payoff. "Manual triage of the viral list against my machine: {{mm}} minutes with a stopwatch. TwoToe: {{s}} seconds while I was asleep. Then I fixed {{k}} things before breakfast." Plus the Temporal crash clip ("I killed it mid-thought. It finished anyway. Nothing ran twice.").

Prize Categories. Only genuine ones: Gemma, Temporal, Sentry Agent Tracing, `.tech` domain, and the chosen host if its category text fits.

Credits and sources. Link every primary source from Appendix D.

17.3 Writing rules for the post

- Every number traces to `RESULTS.md` or a linked source. No exceptions.
- Do not repeat the viral post's inflated claims as fact. Quote it only to correct it.
- Short paragraphs, one idea each. Headers that are sentences, not labels.
- One joke per section, max. The sloth carries the brand; you carry the credibility.
- Read it out loud once. Cut 15%.

18. Demo Video Script (60 to 90 seconds)

Time	Visual	Voice-over / caption
0:00	Viral "internet broke today" post scrolling	"Tuesday: ten scary headlines. Which ones hit my laptop?"
0:07	Terminal: `twotoe sleep`, schedule created	"One command. Then I went to bed."
0:12	Temporal UI, run starts at 03:00 (fixture clock)	"At 3 a.m. a local Gemma reads the findings."
0:20	Split: worker killed with `taskkill /F`, then restarted, workflow completes	"I killed it mid-thought. It finished anyway."
0:32	Windows toast at 07:00 with Line 1	"7 a.m.: two lines."
0:38	Card on phone: checklist, confidence chips, vendor links	"A short checklist. Every item links to the vendor."
0:50	Egress ledger: only localhost (+ cisa.gov)	"Zero secrets left this machine."
0:58	Validator rejection in Sentry trace	"When the model invented a version, the validator threw it out."
1:08	Sloth + `twotoe.tech`	"TwoToe. Slow and steady. Patch, rotate, go back to sleep."

Use fixtures only. No real paths on screen.

19. Risk Register

Risk	Likelihood	Impact	Mitigation
Advisory facts change before Oct 25 (KEV adds, new fixed versions)	High	Medium	`verifiedAt` per entry; re-verify on publish day; post dates its pack
Gemma 4 tag unavailable, or slow on a CPU-only laptop	Medium	High	`TWOTOE_MODEL` fallback; E2B variant; template mode always works
Temporal setup eats time	Medium	Medium	`--no-temporal` Task Scheduler path built in Phase 4 anyway; crash test is the only Temporal must
Sentry docs differ from this spec	High	Low	Phase 7 prompt makes Claude Code read current docs first
Scope creep into stretch goals	High	High	Hard rule: no stretch until the post draft exists
Repeat-winner headwind	Medium	Medium	Aim for Overall via writing; claim only genuine categories
Someone else ships "breach summary bot"	Medium	Medium	Your differentiators: validator, egress ledger, fact-check, crash test, real numbers
Windows path/shim/encoding bugs eat a day	Medium	High	Windows-first CI from Phase 0; rules in 5.4; fixtures use FixtureProbe so the eval never depends on Windows quirks
Real-machine scan reveals something embarrassing	Low	Low	Post aggregate counts only; fix it, then say "I fixed {{k}} things"
Post-deadline commit	Low	Fatal	Tag, then stop pushing; note any later commits in README

20. Final Submission Checklist

Code

- ☐ Repo created inside the window, MIT license, README with install, usage, privacy rules, and "built for DEV Hacktoberfest Week 3"
- ☐ Credits section for any reused open-source code
- ☐ All MUST gates passed; tests green in CI on patched Node
- ☐ `docs/VERIFY.md` empty or every open item explicitly marked "unverified" in the pack and the post
- ☐ Tag `v1.0-submission` pushed before the deadline

Demo

- ☐ Hosted card live on the `.tech` domain, HTTPS, works at 360px
- ☐ Three example briefs, no real paths
- ☐ GIF, crash-test clip, Sentry trace screenshot, card-on-phone screenshot

Post

- ☐ Every template section filled, including My Agent Session and The Lazy Payoff
- ☐ Every number traceable to RESULTS.md or a source
- ☐ Fact-check table included, viral claims corrected, Rajasthan item explained
- ☐ Prize categories: only genuine, re-checked against the category text
- ☐ `#devchallenge` and `#hf26challenge` tags present
- ☐ Published before **Oct 26, 12:29 PM IST** (aim for Oct 25 night)

Appendix A: CLAUDE.md (copy into repo root)

```
# TwoToe: working agreement for Claude Code

Spec: docs/SPEC.md (authoritative). If code and spec disagree, ask before changing the spec.

## Product in one line
Local overnight breach-week triage. Code owns facts, the local model only phrases them,
a validator rejects anything the model said that code did not establish.

## Hard rules (never break, never "temporarily" break)
1. Never read, store, log, trace, or print env VALUES. Key names only. Canary
   TWOTOE_CANARY_9f3a must never appear in any output.
2. Never open IOC files (cook.pth, cook.py, sitecustomize.py) as text, never import or
   execute them, never run Python with user site-packages. stat + sha256 only.
3. Never run pip/npm/pnpm install or any command that executes user project code
   during a scan. execFile only, no shell strings, 5s timeouts.
4. All network goes through the egress guard. Allowed: loopback, www.cisa.gov, and the
   Sentry host only when telemetry is opted in. Add nothing else without asking me.
5. No exploit code, no PoCs, no probing hosts. Defensive only.
6. Demo material and eval use fixtures only. Never commit real paths from my machine.
   Redact C:\Users\<name> (and /Users/, /home/) to ~ in every output.
7. Never change eval/fixtures/*/expected.json to make metrics pass. Fix the code.
8. Facts in packages/core/src/pack/advisories.json must come from a source URL you
   actually fetched. Set verifiedAt. If unsure, write VERIFY and list it in docs/VERIFY.md.

## Platform: Windows first
- I develop and demo on Windows (PowerShell / Windows Terminal), not WSL.
- No bash scripts. All scripts are TypeScript run with tsx.
- path.join everywhere; fast-glob with forward-slash patterns; CRLF-tolerant parsers.
- execFile for .exe, cross-spawn for .cmd shims (pnpm, npm, railway, opencode). Arg arrays only.
- PowerShell: -NoProfile -NonInteractive, user values via env vars, never interpolated.
- Scanners skip EBUSY/EPERM/EACCES files and count them instead of crashing.
- CI must pass on windows-latest.

## Style
- TypeScript strict, zod at boundaries, pure functions in core/match and writer/validate.
- Vitest for everything. Run `pnpm -r test` before saying a phase is done.
- No em dashes in user-facing strings or docs.
- User-facing copy is calm and short. No hype words unless severity rules allow them.

## Workflow
- Work in the phases from SPEC section 16. Stop at each gate and summarize:
  what changed, test results, open questions, anything you could not verify.
- Prefer small commits with clear messages. Never push after tag v1.0-submission.
```

Appendix B: Kickoff Prompt (paste into Claude Code)

```
You are building TwoToe for the DEV Hacktoberfest Week 3 challenge. The full spec is in
docs/SPEC.md (PDF copy at docs/SPEC.pdf) and the working agreement is CLAUDE.md.

Read CLAUDE.md, then SPEC sections 0 through 7 carefully. Before writing any code:
1. Summarize the product, the hard rules, and the phase plan back to me in under 200 words.
2. List every VERIFY item you see and how you plan to check each (which URL).
3. List any spec ambiguity you want me to decide.

Then wait for my "go". After "go", execute Phase 0 exactly as written in section 16,
stop at Gate 0, and report. Do not start Phase 1 until I approve.

Ground rules: never invent advisory facts, never weaken a hard rule to make a test pass,
and tell me plainly when something failed.
```

Useful follow-up prompts during the build:

- "Gate check: run all tests, then list every hard rule from CLAUDE.md and show me the test that enforces it. If a rule has no test, write one."
- "Adversarial review: act as a skeptical DEV judge. Read README and RESULTS.md and list the 5 claims you would doubt. Then show me the evidence for each, or tell me to cut the claim."
- "Privacy sweep: grep the whole repo, ~/.twotoe, and all eval outputs for the canary, absolute home paths, and anything that looks like a key value. Report findings."
- "Post fact-check: for every number and date in docs/post/draft.md, cite the line in RESULTS.md or the source URL. Flag anything unsupported."

Appendix C: Glossary (for the post, and for future you)

Term	Meaning here
Advisory pack	The curated, dated JSON of incidents TwoToe checks
Finding	A matched advisory with evidence, confidence, and priority
Allowed tokens	The exact versions, CVE IDs, vendor names, and numbers the model may say for a finding
Grounding validator	Code that rejects model text containing anything outside allowed tokens
Template mode	Fully deterministic brief with no model call
Egress ledger	Record of every outbound network request during a run
KEV	CISA Known Exploited Vulnerabilities catalog
IOC	Indicator of compromise, here filenames dropped by the malicious LiteLLM release
catchupWindow	Temporal schedule policy that runs a missed 3 a.m. job when the laptop wakes

Appendix D: Sources (verify before publishing)

Challenge

- Challenge page: dev.to/challenges/hacktoberfest-week3-2026-10-19
- Launch post: dev.to/devteam (Join the Hacktoberfest Open-Source AI Challenge: Slow and Steady)
- Hub: dev.to/challenges/hf26 and tag dev.to/t/hf26challenge
- Week 1 winners: dev.to/devteam (Congrats to the ... Touch Grass Winners)
- Weekend winners: dev.to/devteam (Congrats to the ... Build for a Friend Winners)
- Credits: hacktoberfest.com/my/promos

Incidents

- Railway: blog.railway.com/p/security-disclosure-wednesday-october-21st
- Node.js: nodejs.org/en/blog/vulnerability/october-2026-security-releases
- PHP: CISA alert "Critical Vulnerabilities in PHP Require Immediate Patching" (Oct 21, 2026) and php.net release notes for 8.5.12 / 8.4.27 / 8.3.36 / 8.2.35
- GitLab: GitLab patch release post for CVE-2026-110264; Rapid7 analysis of CVE-2026-97229
- Plex2Shell: CISA KEV entries CVE-2026-96652, -96654, -96655, -96656; Plex release notes 1.43.3.10861
- LiteLLM: Wiz, SafeDep, and Socket write-ups on 1.107.0
- eMudhra: Mozilla Bugzilla bug 2083449; Scott Helme "Eight Bits of Trust"; BleepingComputer coverage
- OpenCode: OpenCode's Oct 17 disclosure; Axios coverage
- SBI: SBI's Regulation 30 filing to the stock exchanges (Oct 21, 2026)
- CISA KEV JSON feed: www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json

Docs to read during the build

- Ollama API (structured outputs), Temporal TypeScript SDK (schedules, testing), Sentry AI agent monitoring (Node, manual instrumentation), Mastra (agents, tools, Ollama provider), Vite, Tailwind.

> **Last word.** The judges will forgive a missing stretch goal. They will not forgive a vague post. Freeze early, measure honestly, write it like you talk. Slow and steady.